

QWeave: Correctness-First Deterministic and Learned Qubit Routing

QWeave Research Team

21 September 2026

Abstract

Limited two-qubit connectivity requires a compiler to assign logical qubits to physical sites and route nonlocal interactions. These choices can add SWAPs and circuit depth, while an apparently low-cost circuit is useful only if it preserves the intended computation. This paper develops a correctness-first evaluation of QWeave's current deterministic core: a weighted interaction-graph mapper, strict local improvement, and a shortest-path SWAP router. Identity mapping with the same router, Qiskit SABRE, and a small exact initial-mapping objective oracle provide distinct reference points. We specify route-replay, legality, and small unitary-layout checks, as well as a separately measured dependency-preserving scheduler. We add a guarded Gymnasium environment, a masked graph actor-critic, and a compact PPO implementation with deterministic fallback. A frozen 30-case synthetic suite separates training, validation, and four held-out family-topology test cases. Across 64 test records, every output passed the small-unitary semantic check. The learned, no-message-passing, and untrained controls had identical aggregate median depth and SWAP count, so the study does not support attributing an improvement to learning.

Introduction

Near-term devices constrain which physical qubit pairs can perform two-qubit operations. A logical circuit that ignores this graph requires a physical assignment and, where necessary, SWAP insertion before its gates can execute. Layout and routing are related but distinct compilation stages [5]. SABRE uses bidirectional heuristic search to improve layout and SWAP selection [1]. It is therefore an essential baseline, rather than a straw-man comparator.

QWeave currently supplies a deterministic reference path that makes the mapping convention, tie-breaking, layout updates, and metric extraction explicit. The research question is whether its weighted static mapping surrogate or guarded learned router helps on defined circuit families once the routed circuits are checked for legality and semantics. The scheduler is measured on each fixed legal route, while learned routing is evaluated through separate message-passing and training controls. Existing reinforcement-learning and graph-guided routing studies [2, 3] already establish that learning for this problem is not a novelty claim by itself. This manuscript's contribution at this stage is a precise implementation account and an auditable comparative protocol, not a proven improvement.

Related Work

SABRE combines an initial layout search and SWAP-based routing, including forward-backward circuit traversal [1]. In Qiskit, layout and routing may both change the circuit and add idle physical sites [6]. Pozzi et al. studied deep reinforcement learning for qubit routing with depth overhead as an objective [2]. Sinha et al. combined a graph neural network with Monte Carlo tree search, rather than PPO, for depth-oriented routing [3]. Nannicini et al. formulated joint assignment and routing through integer programming on small instances [4]. QWeave's CP-SAT component has a narrower role: it optimizes only the static initial-mapping surrogate. It cannot certify optimal SWAP count or compiled depth.

Problem and Deterministic Method

Let C be an ordered circuit on logical qubits $L=0,\dots,n-1$ and $H=(P,E,H)$ an undirected hardware graph. The initial injective

mapping $m:L \rightarrow P$ uses the convention $m(\text{logical})=\text{physical}$. The interaction graph GC contains all logical qubits, including isolates; an edge (i,j) has weight w_{ij} equal to its two-qubit interaction count in the default experiment. The static surrogate is

$$J(m) = \sum_{(i,j) \in E_C} w_{ij} d_H(m(i), m(j)). \quad (1)$$

where d_H is shortest-path distance. An optional decay weights an interaction by γ^t for its circuit-operation index t , but the present smoke comparison uses integer counts without decay.

The mapper seeds a high weighted-degree logical qubit on a high-centrality physical node. It then assigns remaining logical qubits greedily according to interaction weight with already mapped neighbours and partial distance cost. Deterministic order resolves ties. Pairwise local search exchanges assigned physical sites, selects the best pair, and accepts a move only if J strictly decreases. Since the set of injective assignments is finite, strict descent terminates; the implementation also caps iterations.

For each source operation in order, the fallback router finds a deterministically chosen shortest hardware path if its two mapped qubits are nonadjacent. It moves the first logical qubit toward the second by edge-local SWAPs, updates both the forward and inverse layout after every inserted SWAP, and emits the operation when adjacent. The final layout is returned alongside the physical circuit. Static distance in Equation (1) is a heuristic for future routing cost: a lower value does not imply fewer SWAPs, minimum legal depth, or higher physical fidelity.

The deterministic Qiskit adapter accepts unconditioned one- and two-qubit gates without classical bits and preserves the source global phase. It rejects measurements, resets, barriers, higher-arity gates, and dynamic operations rather than silently passing unsupported semantics through the router. A trace with one event per emitted operation is replayed to check source gate order, inserted SWAP layouts, and the final mapping. SABRE is given both orientations of each undirected hardware edge so the two paths use the same coupling interpretation. On equal-width unitary circuits of at most six qubits, Qiskit's exposed initial/final layouts support the same small-unitary check; wider or idle-site cases remain unassessed.

The Basic reference uses identity mapping with this same router. The SABRE reference uses Qiskit transpilation at

optimization level zero and a stated seed. CP-SAT minimizes an integer-scaled version of Equation (1) for at most eight logical qubits; only an OPTIMAL solver status certifies that scaled mapping objective. These comparisons isolate the layout choice from the shared deterministic router, while SABRE is a separate compiler path.

Post-Routing Scheduling

The implemented list scheduler accepts an already legal physical circuit. It constructs a dependency graph by preserving operation order on each physical qubit. For operation i with positive duration d_i , the remaining critical-path rank and earliest start are

$$r_i = d_i + \max_{\text{successors } j \text{ of } i} r_j; \quad s_i = \max_{\text{predecessors } j \text{ of } i} (s_j + d_j). \quad (2)$$

with an empty maximum defined as zero. Ready operations are prioritised by r_i , with source order resolving ties; disjoint work can share a start time. The output preserves every physical wire's gate order, global phase, and operation multiset. It does not insert SWAPs or use unproved gate commutations. Therefore its effect is measured as scheduled depth minus routed depth on the same circuit, never as a comparison between two different mappings. Under unit durations, Qiskit's circuit depth already reflects this dependency parallelism, so a zero depth change is expected. Positive per-gate durations additionally yield a timed makespan but do not constitute a calibrated hardware result.

Guarded Learned Router

The learned path is isolated from the deterministic compiler. Its Gymnasium environment exposes an Execute action and one SWAP action for each sorted hardware edge. The observation contains five features per physical node, four features per hardware edge, a five-value frontier description, the current logical-to-physical mapping, and a binary legal-action mask. Illegal actions do not mutate the circuit or layout. If the policy repeatedly fails to make progress, or reaches the episode limit, the environment routes the current operation with the validated deterministic shortest-path method. Every completed episode is replay-validated before a routing result is returned. This guard ensures termination and legality on connected undirected hardware with contiguous labels, but may hide a weak policy behind fallback; fallback counts are therefore reported.

The actor-critic encodes node features and performs two rounds of edge-conditioned message passing. It scores every hardware-edge SWAP and the Execute action after applying the legal-action mask. PPO uses clipped policy ratios, generalized advantage estimation, value and entropy terms, and gradient clipping. We train three explicit seeds. A zero-message-passing PPO control retains the optimizer and reward, while an untrained graph control retains the architecture without PPO updates. All learned variants begin with the weighted deterministic initial mapping, which makes this a routing-policy comparison rather than a learned-layout experiment.

Validation and Experimental Design

The structural validator checks mapping completeness, injectivity, source gate replay, inserted SWAP consistency, global phase, and undirected edge legality for every two-qubit emitted operation. Edge legality alone cannot establish that the logical unitary was preserved. For equal-width small unitary circuits, the test oracle constructs the permutations induced by initial and final layouts and checks

$$U_{\text{routed}} P_{\text{initial}} = P_{\text{final}} U_C. \quad (3)$$

Numerical tests cover mixed one- and two-qubit gates, a source SWAP, and reject an incorrectly reported final layout. This is not yet a general semantic validator for measurements, resets, dynamic conditions, directed couplers, or wider hardware. Those cases will be excluded until explicitly supported.

Benchmark version 1.0 freezes source circuits and graph edge lists before evaluation. It contains 30 synthetic four- and six-qubit cases from five families on connected path, ring, and two-row-grid graphs. Twelve training cases use GHZ, layered-CX, and seeded-interaction families on path and ring; 14 validation cases cover grid transfer and held-out families on path/ring; four untouched test cases combine the held-out star and distant-pair families with grid hardware. Every method receives the same source instance and coupling interpretation. We record source/compiled depth, depth overhead, explicit SWAP count, operation counts, runtime, validity, layouts, seeds, versions, and hashes. Source depth is a reference, not an optimal legal depth. SABRE uses five stated seeds; learned training uses three. Runtime is the median of three measurements after one warmup on the same machine. Seeds are collapsed within each case before the four cases are aggregated. Scheduling is reported as a within-route stage delta; CP-SAT reports only its scaled mapping objective. The suite is too small for population-level or state-of-the-art claims.

Results and Limitations

A fresh local run passed 52 tests under Python 3.11.15 and Qiskit 2.5.2. Table 1 reports the frozen four-case test aggregate. For seeded methods, each case contributes its within-case seed median before the median over cases. All 64 compiled outputs passed the small-unitary layout check. Every unit-duration scheduling delta was zero.

Table 1 Held-out test summary. Runtime is median wall time in seconds.

Method	Depth	SWAP	Runtime	Fallback
Basic	9.5	3.0	0.0085	0.0
Weighted	7.5	1.5	0.0083	0.0
SABRE	10.0	2.5	0.0052	0.0
GNN-PPO	7.5	1.0	0.0588	0.0
No-GNN PPO	7.5	1.0	0.0260	0.0
Untrained GNN	7.5	1.0	0.0594	0.0

The aggregate GNN-PPO values are numerically lower than Basic and SABRE on this small suite, but this cannot be attributed to the GNN or PPO: both controls have the same aggregate depth and SWAP medians. The untrained control also

contains severe seed-specific outliers and invoked deterministic fallback 16 times across its 12 records, whereas the two trained variants used no test fallbacks. GNN-PPO inference was about seven times the median weighted-router runtime in this Python implementation. These observations motivate larger training budgets, more test instances, confidence intervals, named algorithm circuits, and hardware-directed studies. They do not establish routing superiority, novelty, or production readiness.

Conclusion

QWeave now combines an explicit deterministic reference, a small mapping oracle, fixed-route scheduling, and a guarded learned router with auditable ablation controls. The frozen v1 run establishes implementation validity on its declared small-unitary scope, while showing that the present aggregate results do not isolate a benefit from message passing or PPO. The next scientific gate is a larger preregistered study with stronger training, additional circuit families, confidence intervals, and independent reruns.

References

- [1] G. Li, Y. Ding, and Y. Xie. Tackling the Qubit Mapping Problem for NISQ-Era Quantum Devices. ASPLOS, 2019. arXiv:1809.02573.
- [2] M. G. Pozzi, S. J. Herbert, A. Sengupta, and R. D. Mullins. Using Reinforcement Learning to Perform Qubit Routing in Quantum Compilers. arXiv:2007.15957, 2020.
- [3] A. Sinha, U. Azad, and H. Singh. Qubit Routing using Graph Neural Network aided Monte Carlo Tree Search. AAAI, 2022. arXiv:2104.01992.
- [4] G. Nannicini, L. S. Bishop, O. Gunluk, and P. Jurcevic. Optimal Qubit Assignment and Routing via Integer Programming. ACM Transactions on Quantum Computing, 2022.
- [5] IBM Quantum. Transpiler stages. IBM Quantum Documentation, 2026.
- [6] IBM Quantum. SabreLayout API reference. IBM Quantum Documentation, 2026.